

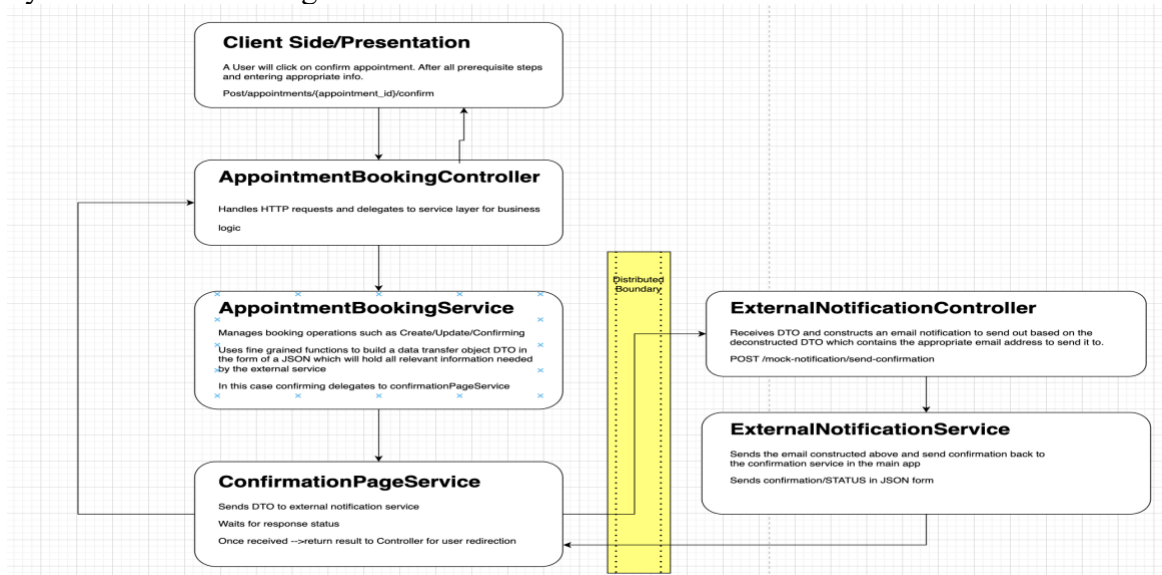
Michael Fox

CMPE 172

22, April 2026

Term Project M5

1. Distributed Boundary Definition: The boundary is the point where information is sent or received between processes instead of just normal within process method calls. These external calls take much more time than internal.
2. System Interaction Diagram:



3. External Service Description: Service will receive the AppointmentConfirmationDTO and construct and email to patient as way of notifying the patient of a Confirmation of appointment. It needs to be able to extract relevant details from DTO, construct email. Send Email. Then return a confirmation JSON to main app.
4. REST API Design:

```
@service no usages
public class ConfirmationPageService {
    private final RestTemplate restTemplate = new RestTemplate(); 1usage

    public NotificationResponse sendConfirmation(AppointmentConfirmationDTO dto) { no usages

        String url = "http://localhost:8080/mock-notification/send-confirmation";

        ResponseEntity<NotificationResponse> response =
            restTemplate.postForEntity(
                url,
                dto,
                NotificationResponse.class
            );

        return response.getBody();
    }
}
```

The ConfirmationPageService creates restTemplate which will handle the post using the url, the DTO, and waits for a response JSON. Then converts JSON to java object. The response JSON should look like:

```
{
  "status": "SENT",
  "messageId": "MSG-001"
}
```

Indicating the confirmation email was sent, and the corresponding messageId.

The DTO should look something like:

```
public AppointmentConfirmationDTO(Long appointmentId, String patientName, no usages
                                String patientEmail, String doctorName,
                                String appointmentTime) {

    this.appointmentId = appointmentId;
    this.patientName = patientName;
    this.patientEmail = patientEmail;
    this.doctorName = doctorName;
    this.appointmentTime = appointmentTime;
}
```

This object is created using the AppointmentBookingService using fine grained in process local method calls to the DB. By creating this DTO and sending it to the external service we are only sending one object containing all the data the service needs to to send the confirmation email. This is utilizing the Coarse Grained method as it avoids making external calls multiple times unnecessarily.

5. Mock Service Implementation:

The external service will utilize @PostMapping("/mock-notification/send-confirmation") in the external notification controller. This will then call the connected service to send the email. Then returns the response that is a status and msgId to the controller.

The controller then uses return new NotificationResponse(response);

6. Design Justification: This way of doing things separates the notification delivery from appointment creation. Separation of responsibilities is similar to real world scaling and applications. The main app here will handle appointment related creation/deletions/updating/etc. Where it can delegate the notification system to another service allowing it to specialize in its own domain area. Also by utilizing a coarse grained solution we are only sending one 'dto' as a Json to the external service, this prevents the many costly back and forth calls to get all required information.